

CV7: Navigation Compose v Jetpack Compose - Teória

1 Cieľ cvičenia

V tomto cvičení sa budeme venovať navigácii medzi obrazovkami v Jetpack Compose pomocou knižnice Navigation Compose. Po absolvovaní cvičenia by ste mali rozumieť tomu, ako sa v Compose vytvára jednoduchá viacobrazovková aplikácia, ako sa definujú jednotlivé obrazovky a ako sa medzi nimi odovzdávajú jednoduché dáta.

Výsledkom bude miniaplikácia typu Recipe Browser. Používateľ bude vedieť prezerat' zoznam receptov, otvoriť detail konkrétneho receptu a vrátiť sa späť na zoznam. Aplikácia je zvolená zámerne, pretože veľmi prirodzene ukazuje prechod zo zoznamu na detail a zároveň ostáva dostatočne jednoduchá na sústredenie sa na samotnú navigáciu.

2 Čo je Navigation Compose

Navigation Compose je knižnica, ktorá slúži na navigáciu medzi obrazovkami v Compose aplikácii. Namiesto toho, aby bola aplikácia tvorená iba jednou veľkou composable funkciou, môžeme ju rozdeliť na viacero samostatných obrazoviek a definovať, ako sa medzi nimi prechádza.

V tomto cvičení bude mať aplikácia dve obrazovky. Prvá zobrazí zoznam receptov a druhá detail vybraného receptu. Používateľ klikne na recept v zozname a aplikácia sa presunie na detail. Navigácia je základná súčasť architektúry aplikácie, pretože určuje, ako medzi sebou jednotlivé obrazovky súvisia.

3 Dependency pre Navigation Compose

Aby bolo možné Navigation Compose v projekte používať, je potrebné pridať dependency do súboru `build.gradle.kts` na úrovni modulu aplikácie.

```
implementation("androidx.navigation:navigation-compose:2.9.0")
```

Po pridaní dependency je potrebné projekt zosynchronizovať. Bez tejto knižnice nebudú dostupné composable funkcie ako `NavHost`, `composable(...)` ani funkcia `rememberNavController()`.

4 Základné pojmy

Pri práci s navigáciou sa používajú najmä nasledujúce pojmy.

- **NavController** je objekt, ktorý riadi navigáciu medzi obrazovkami.
- **NavHost** je kontajner, v ktorom sa nachádza navigačný graf aplikácie.
- **Route** je textový identifikátor obrazovky.
- **Composable destination** je konkrétna obrazovka reprezentovaná composable funkciou.

Tieto pojmy sa v tomto cvičení objavajú spolu. Nemali by ste vnímať **NavController** alebo **NavHost** ako izolované prvky, ale ako časti jedného celku.

5 NavController

NavController je hlavný objekt, ktorý riadi navigáciu medzi obrazovkami. V **Compose** sa vytvára pomocou funkcie `rememberNavController()`.

```
val navController = rememberNavController()
```

Tento objekt sa následne používa pri prechode na inú obrazovku.

```
navController.navigate("detail/1")
```

Je užitočné predstaviť si **NavController** ako komponent, ktorý rozhoduje, ktorá obrazovka sa má zobraziť a aké argumenty sa do nej majú odovzdať.

6 NavHost

NavHost predstavuje kontajner, v ktorom je definovaný navigačný graf aplikácie. Určuje, aké obrazovky aplikácia obsahuje a ktorá z nich sa otvorí ako prvá.

```
NavHost(  
    navController = navController,  
    startDestination = Routes.LIST  
) {  
    composable(Routes.LIST) {  
        RecipeListScreen(navController)  
    }  
  
    composable(  
        route = Routes.DETAIL + "{recipeId}"  
    ) { backStackEntry ->  
        RecipeDetailScreen(...)   
    }  
}
```

Parameter `startDestination` určuje počiatočnú obrazovku aplikácie. V našom prípade to bude obrazovka so zoznamom receptov.

7 Routes

Dobrou praxou je definovať routes centrálne na jednom mieste. Tým sa znižuje pravdepodobnosť chýb spôsobených preklepom a kód je prehľadnejší.

```
object Routes {  
    const val LIST = "list"  
    const val DETAIL = "detail"  
}
```

8 Dátový model aplikácie

Aj v tomto cvičení je dôležité postaviť aplikáciu okolo jednej hlavnej entity. V našom prípade je touto entitou `Recipe`. Každý recept bude obsahovať identifikátor, názov, krátky popis, ingrediencie a postup.

```
data class Recipe(  
    val id: Int,  
    val name: String,  
    val description: String,  
    val ingredients: List<String>,  
    val instructions: String  
)
```

Takýto model je zámerne jednoduchý. Mali by ste sa vďaka tomu môcť sústrediť na navigáciu, nie na komplikovanú logiku. Zároveň je však model dostatočne bohatý na to, aby sa dal na detailovej obrazovke zobraziť zmysluplný obsah.

9 Vzorové dáta

Dáta pripravte priamo v aplikácii ako jednoduchý zoznam receptov. Nemusíte sa zaoberať databázou ani ViewModelom, preto celú pozornosť sústreďte na navigáciu.

Príklad definície vzorových dát môže vyzeráť takto:

```
object RecipeData {  
    val recipes = listOf(  
        Recipe(  
            id = 1,  
            name = "Spaghetti Carbonara",  
            description = "Classic pasta with creamy sauce.",  
            ingredients = listOf("Spaghetti", "Eggs", "Parmesan")  
            ,  
            instructions = "Cook the pasta and mix all  
                           ingredients."  
        )  
    )  
}
```

Okrem samotného zoznamu je potrebné pripraviť aj pomocnú funkciu, ktorá vyhľadá recept podľa identifikátora.

```
fun getRecipeById(id: Int): Recipe? {  
    return recipes.find { it.id == id }  
}
```

10 Architektúra miniaplikácie

Aplikácia Recipe Browser bude obsahovať dve obrazovky.

Prvá obrazovka bude `RecipeListScreen`. Na tejto obrazovke sa zobrazí zoznam receptov pomocou `LazyColumn`. Každá položka bude reprezentovaná kartou s názvom a krátkym popisom.

Druhá obrazovka bude `RecipeDetailScreen`. Na tejto obrazovke sa zobrazí detail vybraného receptu, teda názov, popis, ingrediencie a postup.

Tok aplikácie bude nasledovný. Používateľ otvorí zoznam receptov. Kliknutím na kartu receptu sa vykoná navigácia na detail. Pri navigácii sa odošle parameter `recipeId`. Detailová obrazovka tento parameter načíta, vyhľadá správny recept a zobrazí jeho obsah. Následne sa používateľ môže vrátiť späť na zoznam.

11 Obrazovka so zoznamom receptov

Zoznam receptov bude implementovaný pomocou `LazyColumn`. To je prirodzené nadviazanie na predchádzajúce cvičenie, v ktorom ste pracovali so zoznamami.

```
LazyColumn {
    items(recipes, key = { it.id }) { recipe ->
        RecipeCard(
            recipe = recipe,
            onClick = { /* ... */ }
        )
    }
}
```

Každá položka v zozname bude klikateľná. Po kliknutí sa vykoná navigácia na detailovú obrazovku.

```
navController.navigate(
    Routes.DETAIL + "${recipe.id}"
)
```

12 Odovzdanie parametra

Pri navigácii medzi obrazovkami nie je vhodné posilať celý objekt receptu. V tomto cvičení sa bude medzi obrazovkami odovzdávať iba jednoduchý parameter `recipeId`.

Výhodou tohto prístupu je, že route ostáva jednoduchá, čitateľná a dobre sa nadväzuje na ďalšie témy, napríklad ViewModel alebo perzistenciu dát.

Navigácia na detail môže vyzeráť takto:

```
navController.navigate(
    Routes.DETAIL + "${recipe.id}"
)
```

Na cieľovej obrazovke sa parameter načíta z argumentov:

```
val recipeId =
    backStackEntry.arguments
        ?.getString("recipeId")
        ?.toIntOrNull()
```

Tento parameter sa následne použije na vyhľadanie konkrétneho receptu vo vzorových dátach.

13 Detailová obrazovka

Detailová obrazovka prijme identifikátor receptu a pomocou neho vyhľadá správny objekt.

```
val recipe = recipeId?.let {  
    RecipeData.getRecipeById(it)  
}
```

Ak sa recept nájde, zobrazí sa jeho obsah. Ak sa recept nenájde, je vhodné zobrazit jednoduchú informačnú správu.

Na detailovej obrazovke sa budú zobrazovať štyri základné časti:

- názov receptu,
- krátky popis,
- zoznam ingrediencií,
- postup prípravy.

14 Návrat späť

Na návrat na predchádzajúcu obrazovku sa používa funkcia `popBackStack()`.

```
navController.popBackStack()
```

Táto funkcia odstráni aktuálnu obrazovku zo zásobníka navigácie a používateľ sa vráti na predchádzajúcu obrazovku. V našom prípade sa teda z detailu vráti späť na zoznam receptov. Taktiež je potrebné vedieť, že navigácia nie je jednosmerná. Pri návrhu viacobrazovkovej aplikácie treba myslieť aj na návrat používateľa.

15 Potrebné importy

Pri implementácii budú dôležité najmä nasledujúce importy:

```
import androidx.navigation.NavHostController  
import androidx.navigation.compose.NavHost  
import androidx.navigation.compose.composable  
import androidx.navigation.compose.rememberNavController
```

Tieto importy patria ku klasickej Navigation Compose knižnici. Ak niektorý z nich projekt nepozná, treba najskôr overiť, či bola správne pridaná dependency do `build.gradle.kts` a či bol projekt zosynchronizovaný.

16 Rozdelenie aplikácie na samostatné časti

Aj pri navigácii je vhodné rozdeliť aplikáciu na menšie časti. V tomto cvičení bude vhodné vytvoriť minimálne tieto súbory alebo logické časti:

- MainActivity,
- NavigationGraph,
- Routes,
- Recipe,
- RecipeData,
- RecipeListScreen,
- RecipeDetailScreen.

17 Čo si máte z cvičenia odniesť

Po zvládnutí tohto cvičenia by ste mali rozumieť tomu, ako vytvoriť jednoduchú viacobrazovkovú Compose aplikáciu. Zároveň by ste mali vedieť, prečo sa pri navigácii odovzdáva jednoduchý parameter namiesto celého objektu a ako sa tento parameter použije na vyhľadanie dát.

Toto cvičenie zároveň pripravuje pevný základ pre ďalšie témy, najmä pre ViewModel a oddelenie používateľského rozhrania od dátovej logiky.